

DIP: 面向高效大多模态模型训练的动态交错流水线

Zhenliang Xue

Institute of Parallel and Distributed
Systems
Shanghai Jiao Tong University
Shanghai, China
xuezhengliang@sjtu.edu.cn

Hanpeng Hu

StepFun
Shanghai, China
haanpeng@outlook.com

Xing Chen

StepFun
Shanghai, China
xchen382@asu.edu

Yimin Jiang

StepFun
Shanghai, China
jymthu@gmail.com

Yixin Song

Institute of Parallel and Distributed
Systems
Shanghai Jiao Tong University
Shanghai, China
yixinsong@sjtu.edu.cn

Zeyu Mi

Institute of Parallel and Distributed
Systems
Shanghai Jiao Tong University
Shanghai, China
yzmizeyu@sjtu.edu.cn

Yibo Zhu

StepFun
Shanghai, China
zhuyibo@stepfun.com

Daxin Jiang

StepFun
Shanghai, China
djiang@stepfun.com

Yubin Xia

Institute of Parallel and Distributed
Systems
Shanghai Jiao Tong University
Shanghai, China
xiayubin@sjtu.edu.cn

Haibo Chen

Institute of Parallel and Distributed
Systems
Shanghai Jiao Tong University
Shanghai, China
haibochen@sjtu.edu.cn

Abstract

大多模态模型 (LMM) 已在涉及多种模态的理解与生成任务中展现出卓越能力。尽管这些模型可以接收灵活组合的输入数据, 其训练效率仍受两个主要问题影响: 异构模型架构造成的流水线阶段不均衡, 以及多模态数据多样性所引起的训练数据动态性。

本文提出 DIP, 一个面向 LMM 训练的动态、模态感知流水线调度框架。DIP 通过两项关键技术应对动态不均衡挑战: (1) 将不同模态的计算分离到专用的流水线段中, 从而在一组连续阶段内部平衡工作负载; (2) 将输入数据动态拆分为粒度更细、特定于模态的子微批次, 从而在这些流水线段之间平衡工作负载。训练期间, DIP 利用空闲 CPU 资源异步生成流水线调度, 无需停顿训练过程即可针对每个输入批次动态定制阶段执行。我们在五种多样化的 LMM 上验证了 DIP; 这些模型的参数规模从 12B 到 94B, 涵盖视觉语言模型和扩散模型。实

验结果表明, 与最先进的系统相比, 本系统可将吞吐量最高提升 97.3%, 展现出对波动的多模态训练工作负载的强适应能力。

CCS Concepts: • Computing methodologies → Distributed computing methodologies; Modeling and simulation.

Keywords: 大规模训练; 大多模态模型; 分布式系统; 仿真

ACM Reference Format:

Zhenliang Xue, Hanpeng Hu, Xing Chen, Yimin Jiang, Yixin Song, Zeyu Mi, Yibo Zhu, Daxin Jiang, Yubin Xia, and Haibo Chen. 2026. DIP: 面向高效大多模态模型训练的动态交错流水线. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '26)*, March 22–26, 2026, Pittsburgh, PA, USA. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3779212.3790154>

1 引言

基于 Transformer 的模型 [11, 17, 44, 51] 在多模态理解、推理与生成方面展现出卓越能力, 并已成为下一代大多模态模型 (LMM) [2, 10, 17, 18, 31, 32] 的基础架构。现代 LMM 集成了多个模态模块, 包括通过模态适配器连接的编码器、解码器和骨干模型。这种架构设计能够灵活、无缝地处理交错的多模态数据 (例如文本、图像和



This work is licensed under a Creative Commons Attribution 4.0 International License.

ASPLOS '26, Pittsburgh, PA, USA

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2359-9/2026/03

<https://doi.org/10.1145/3779212.3790154>

视频), 从而支持多模态文档理解 [8] 和多轮对话 [33] 等多样化任务范式。

然而, 这种架构灵活性也给训练带来了重大挑战: 不同模态模块和数据批次的执行延迟既不规则又不断波动, 由此产生了独特的动态不平衡问题。该挑战源于两个相互关联的因素:

流水线阶段不平衡: LMM 包含多种模态模块, 各模块具有不同的参数形状和算子类型 (例如注意力、GEMM 和卷积), 因而呈现出架构异构性。这些差异会造成偏斜的计算负载和不同的内存访问模式, 使模型执行难以被划分为均衡的流水线阶段。即使穷举所有可能的层切分方式, 对于一个 37B 参数的 LMM, “最优”划分仍会产生 22.8% 的流水线气泡开销 (§2.3)。这种低效主要源于异构层之间的显著差异, 使流水线阶段无法实现完全均衡的划分。

训练数据动态性: 多模态训练数据的多样性进一步加剧了架构本身的异构性; 大规模数据集中, 不同批次的模态分布高度可变。尽管数据打包技术 [24, 48] 试图构造更均衡的批次, 计算不平衡仍然存在。在我们的实验中, 最大数据样本带来的计算负载是最小样本的 4.15× (§2.2)。这种差异会导致批次之间出现显著的工作负载波动。

流水线阶段不平衡与训练数据动态性共同形成了显著的性能瓶颈。我们的实验表明, 对于 37B LMM, 这种动态不平衡可使训练开销最高增加 40.3% (§2.3)。面对 LMM 的动态特性, 这种性能下降使采用单一静态流水线调度的传统方法 (例如 Megatron-LM 的 1F1B) 既不切实际又效率低下。同样, Spindle [47] 和 Optimus [15] 等多模态模型训练系统依赖针对固定任务集合定制的静态流水线调度。它们的训练计划在训练开始前即被确定, 忽略了多模态数据的动态性, 因而产生次优的流水线性能。面向动态文本序列长度设计的现有方法 [24, 48, 54] 同样不够充分。它们主要处理会均匀影响单模态 LLM 所有层的工作负载变化。然而, 这些方法无法解决 LMM 中根本性的模态间不平衡: 某一模态发生变化 (例如图像增多) 时, 特定模块可能严重过载, 而其他模块仍未得到充分利用。

为应对上述挑战, 我们提出 DIP, 一个旨在优化 LMM 流水线并行的动态模态感知调度框架。DIP 建立在两个揭示流水线低效根因的关键洞察之上。第一, 由于不同模态模块的计算成本差异显著, 将其计算放在同一次流水线执行过程中本质上是低效的 (图 5a)。我们将流水线阶段定义为跨越所有流水线阶段的一次完整前向或反向计算过程。为消除这种段内不平衡, DIP 强制采用分离式划分, 为每种模态分配彼此独立的流水线段 (图 5b)。这种隔离可防止较慢模态成为较快模态的瓶颈。第二, 即使完成分离, 这些特定于模态的流水线段之间仍存在工作负载不平衡。为解决这种段间不平衡, DIP 进一步将数据动态拆分为更小、特定于模态的子微批次 (图 5c)。这种模态感知划分使不同模态流水线路段的延迟能够更紧密地匹配, 从而在这些段之间实现粒度更细的负载均衡。

与此同时, 与依赖静态或预计算调度的传统系统不同, DIP 采用异步调度生成。它即时为每个数据批次生成定

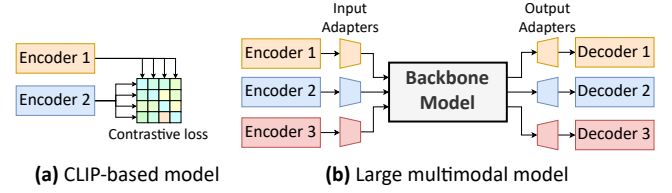


Figure 1. 基于 CLIP 的模型与 LMM 的比较。

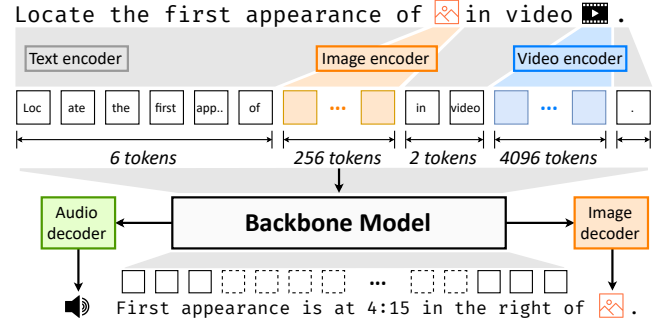


Figure 2. 以大语言模型作为骨干的 LMM 示例。用户提示由一幅图像和一个视频片段组成, 它们经相应模态编码器转换为 token, 随后由骨干模型进一步处理, 以多模态文本或语音音频的形式生成响应。

制流水线调度, 无缝适应多模态数据不断波动的计算需求。为避免训练停顿, DIP 利用空闲 CPU 核心生成调度, 并与主要 GPU 训练工作进程并行运行, 从而有效隐藏搜索延迟。为了在一次训练迭代的严格时限内 (通常为 10–60 秒) 找到高质量调度, DIP 将复杂的搜索问题分解为三个更简单的子问题, 并为每个子问题设计启发式方法以缩小搜索空间。DIP 在数百个 CPU 核心上并行化搜索循环, 既缩短搜索时间, 也利用了空闲 CPU。

我们在面向 Transformer 大语言模型的最先进分布式训练框架 Megatron-LM 之上实现了 DIP。除调度算法外, 我们还开发了训练仿真器, 用于快速、准确地预测流水线阶段延迟和内存消耗; 同时增强 Megatron-LM, 使其具有可重配置流水线机制, 以支持动态部署流水线调度。我们在五种多样化的 LMM 上验证了 DIP; 这些模型参数规模从 12B 到 94B, 涵盖视觉语言模型和扩散模型。实验结果表明, 相比基线系统, DIP 可将训练性能最高提升 97.3%。此外, DIP 对 LMM 训练中的动态工作负载表现出优异适应性, 在整个训练过程中维持接近最优的硬件利用率。

总而言之, 本文作出以下贡献:

- 我们识别并刻画了 LMM 训练中的动态不平衡问题; 该挑战源于流水线阶段不平衡与训练数据动态性的相互作用。
- 我们提出 DIP, 一个采用异步调度生成、模态感知划分和分解式搜索算法的新型调度框架, 可动态适应不断变化的工作负载。
- 我们在五种、最高 94B 的 LMM 上将 DIP 与三个最先进训练系统进行比较, 结果表明训练效率最高提升 97.3%。

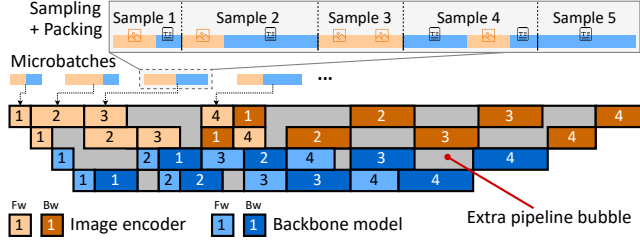


Figure 3. 动态不平衡影响示意图。

2 动态不平衡特征分析

2.1 大多模态模型

多模态模型长期以来一直试图通过学习统一的语义表示来连接异构模态 [40]。早期基于 CLIP 的多模态模型 [37] 开创了双编码器架构（例如图像使用 ViT、文本使用 BERT），并利用对比损失 [6] 在共享语义空间中对齐跨模态嵌入。

大多模态模型（LMM）的出现标志着一次范式转变：其目标从表示对齐扩展到多模态生成能力。这一演进由大规模、基于 Transformer 的架构整合所推动，包括大语言模型（LLM）、视觉 Transformer（ViT）和扩散 Transformer（DiT）。如图 1 所示，LMM 使用模式适配器将编码器与骨干模型集成。随后，骨干模型通过自回归过程或扩散机制生成输出表示，再由专用模式解码器将这些表示转换为文本、音频和图像等人类可感知的形式。与基于 CLIP 的模型相比，LMM 的可扩展生成式架构显著拓宽了所支持任务的范围。例如，在一个 LMM 中（图 2），用户可在同一查询中交错使用文本、图像和视频，并通过追加后续问题反复细化查询，形成多轮对话 [10, 20, 33]。LMM 可以文本或语音形式响应 [18]，还可使用扩散模块生成图像 [27, 31, 35]。

LMM 训练。大型 Transformer 模型通常采用数据并行（DP）、流水线并行（PP）和张量并行（TP）等三维分布式方案进行训练 [39]。流水线并行将模型层划分为多个模型块，并放置在不同机器（即流水线秩）上。每个模型块都可以执行前向和反向阶段计算。数据批次被拆分为更小的微批次，并通过点到点（P2P）通信在流水线阶段之间传递。

2.2 动态不平衡的来源

由于多模态模型架构的异构性和训练数据的多样性，LMM 的训练过程与单模态模型显著不同，主要体现在流水线阶段不平衡和训练数据动态性两个方面。

流水线阶段不平衡。最优流水线效率要求均衡地划分阶段以尽量减少气泡，但模态模块之间固有的计算差异带来了根本性挑战。考虑一个在 H800 GPU 上运行的 37B VLM，它由一个 5B ViT 编码器（64 层）和一个 32B 语言模型（64 层）组成，处理的批次包含 8 幅图像和 8192 个文本 token。每个 ViT 层处理图像需要 6.75ms（前向加反向），而每个 LM 层处理这些 token 需要 10.5ms。在 16 个流水线阶段上穷举搜索得到的最优划分方案，其阶段延迟介于 63ms 和 73.5ms 之间（即相差 16.7%）。采用

Table 1. 7B 模型在 8 块 GPU（TP=2，PP=4）上的训练性能。“PFLOPs”表示每次迭代的浮点运算量（单位为 petaFLOPs），为公平比较，该数值被控制为固定值。“MFU”表示模型 FLOPs 利用率。

模型配置	时间 (s)	PFLOPs	MFU
LM 7B	4.068	12.8	0.400
ViT 2B + LM 5B（静态数据）	4.567	12.7	0.351
ViT 2B + LM 5B（动态数据）	6.789	12.8	0.239

64 个微批次和 Megatron-LM 的 1F1B 流水线调度时，这种不平衡会引入额外 22.8% 的流水线气泡，说明实现完全均衡划分存在固有困难。

训练数据动态性。多模态训练数据的动态异构性使这一挑战进一步加剧：不同模态的批次差异会带来波动的计算需求。多模态数据来自多种来源，包括：图像-描述对 [3, 38]、带字幕的视频 [4, 7, 46, 49]，以及图文交错内容 [25, 58]。数据样本通常由图像或视频及其描述文本组成，不同数据集之间的模态数据比例存在显著的跨数据集差异（图 4a-b）。例如，LAION-2B [38] 数据集由图像和简短字幕配对构成，文本-图像比很小（16.4 token/图像）；而 OBELICS [25] 数据集包含完整的多模态文档，其比例变化范围极大（0.4 到 3115 token/图像）。

在数据打包过程中，这种差异还会引起跨批次不平衡。通常会将数据样本打包成更大的批次，以提高计算效率 [24, 48]。对于视觉语言模型（VLM），图像和文本都会被 token 化（例如一幅图像转为 169 个 patch token），随后以贪心方式打包至模型上下文长度上限（例如 8192 token），形成一个微批次。类似地，对于文本到视频（T2V）扩散模型，通常会把时长和宽高比相近的视频片段分组，以便进行批处理 [31, 35]。然而，由于不同模态的分布存在差异，数据打包无法保证多个模态之间的工作负载均衡（图 4c-d）。例如，考虑两个容量同为 8192 token 的微批次：第一个包含 10 幅图像（即 1690 个 patch token）和 6502 个文本 token，而第二个仅包含一幅图像和 8023 个文本 token。这两个微批次带来的计算需求截然不同：第一个微批次中图像流水线阶段所需的 FLOPs 是第二个的 10×。这种差异导致不同微批次的流水线阶段不平衡（图 3）。此外，对于由 7B LM 和 5B DiT 组成的 T2V 模型，即使经过数据打包，计算最密集的批次（图 4d 中的数据批次 100）的计算负载仍是最小批次（数据批次 1）的 4.15×。

2.3 对流水线并行的负面影响

流水线阶段不平衡和训练数据动态性共同导致了动态不平衡问题。如图 3 所示，该问题在 Megatron-LM 的 1F1B 流水线调度中引入显著气泡，并严重降低训练吞吐量。

为量化这一影响，我们比较了两个参数量相同的模型配置：单模态 LM（7B），以及视觉语言模型（ViT 2B + LM 5B）。在相同的 1F1B 流水线配置下（采用参数量均衡划分并固定计算预算），VLM 在静态数据上因阶段不平衡产生 12.5% 的开销。使用真实世界动态数据时，该开销上升到 40.3%，如表 1 所示。

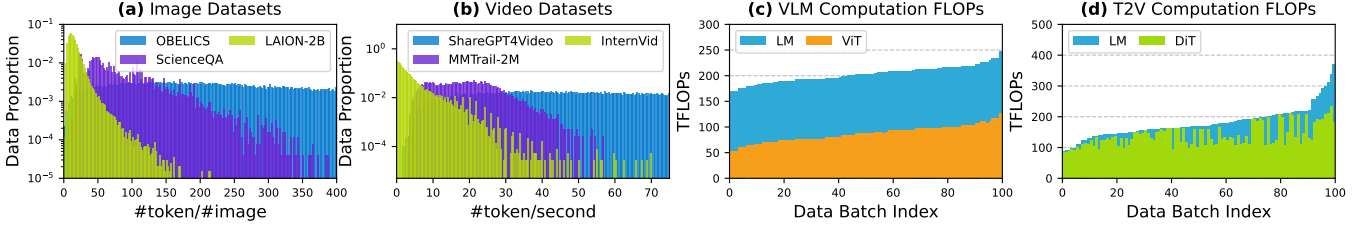


Figure 4. (a–b) OBELICS [25], LAION-2B [38] 和 ScienceQA [30] 数据集中每幅图像对应的 token 分布, 以及 ShareGPT4Video [4], InternVid [46] 和 MMTrail-2M [7] 视频数据集中每秒视频对应的 token 分布。Y 轴表示归一化数据比例。(c–d) VLM 12B (表 3 中的 VLM-S) 和 T2V 13B (表 3 中的 T2V-S) 模型在 100 个已打包数据批次上的计算需求。批次 (X 轴) 按计算成本升序排列, 浮点运算量 (Y 轴) 以 teraFLOPs (TFLOPs) 计量。

对流水线设计的影响。这种性能下降使单模态 LLM 使用的固定流水线调度不再可行。由于每次迭代处理不同的数据批次, 最优流水线调度会动态变化。

预计算流水线调度同样不可行。可能的输入组合数量会沿两个维度指数增长: 模态数量和微批次数量。在我们的实验设置中, 每个微批次最多可包含 48 幅图像 (§7.1); 对于 64 个微批次, 这会形成 $49^{64} \approx 1.5e108$ 种不同输入配置。因此, 不可能为所有场景穷举预计算最优调度。尽管只为输入的某个子集进行预计算可以减少开销, 但从如此庞大的空间中选择具有代表性的子集极具挑战, 而且无法泛化到未见输入。

若干先前工作通过改进数据打包策略来减少微批次之间的差异 [48], 或通过自适应重排微批次来尽量减少不规则输入造成的气泡 [24, 54], 从而解决文本序列长度的动态性问题。然而, 这些方法忽略了模态模块之间的异构性, 因此不足以解决多模态训练中的动态不平衡。在单模态 LLM 中, 序列长度变化会均匀影响所有层; 与之不同, 多模态微批次对特定模态模块的影响并不相同。例如, 增加图像数量会显著提高图像编码器的计算需求, 却几乎不影响语言模型骨干。这种模态间不平衡使面向单模态模型设计、以数据为中心的方法无法有效用于 LMM 训练。

3 方法概览

为应对流水线阶段不平衡和训练数据动态性带来的挑战, 我们提出动态交错流水线 (Dynamic Interleaved Pipeline, DIP), 一个旨在优化 LMM 流水线并行的动态模态感知调度框架。DIP 建立在三项关键设计原则之上: 通过异步生成调度来隐藏搜索延迟, 通过模态感知划分从源头缓解不平衡, 并通过分解式搜索算法高效找到高质量流水线调度。

3.1 设计原则

异步调度生成。与依赖静态或预计算调度的传统方法不同, 为了处理 LMM 数据的动态特性, DIP 采用在线异步策略。对于每个即将处理的数据批次, 它并发、即时地生成定制流水线调度, 从而有效适应多模态数据不断波动的计算需求。这一过程包括数据预取和调度搜索, 并在空闲 CPU 资源上异步运行, 与 GPU 上的主要训练工作进程并行。由于不处于关键路径上, 我们的方法可在

不引入显著开销的情况下获得处理动态数据所需的适应能力。

模态感知划分。如 §2.2 所述, LMM 训练效率低下的主要根源是不同模态模块之间动态变化的计算不平衡。这种动态不平衡会造成不规则的阶段延迟, 使流水线气泡难以消除。为此, DIP 通过尽量缩小阶段延迟差异, 从源头减少流水线气泡。具体做法是将不同模态模块的计算划分到宽度相同的流水线段中。我们的方法由两项关键技术指导:

① 通过分离式划分消除段内不平衡。一个流水线段是一组分布在全部 P 个流水线秩上的连续阶段。例如, 图 5a 中编号为“3”的四个前向阶段构成一个流水线段, 其对应的反向阶段则构成另一个流水线段。我们观察到, 将不同模态模块的阶段放在同一流水线段中 (图 5a) 本质上是低效的。由于视觉阶段和语言阶段的计算成本不同, 而且对数据比例 (例如图像数与 token 数) 的敏感程度也不同, 将它们混合会在流水线秩之间造成不可避免的延迟差异。我们的第一项原则是强制采用分离式划分 (图 5b), 使每个模态模块占用各自专用的流水线段。这消除了流水线气泡的一个主要来源。对于 ViT 2B + LM

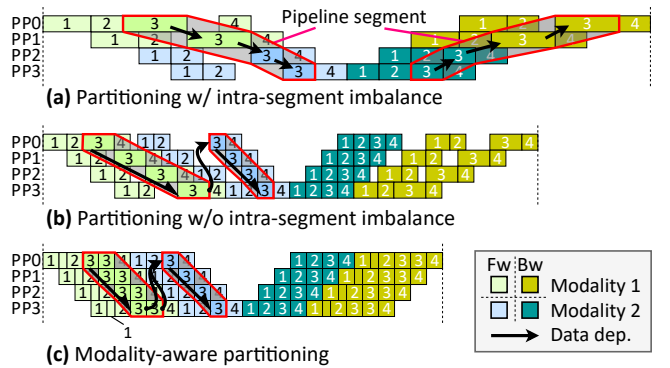


Figure 5. §3.1 中各划分方案的示意图。方框中的数字表示微批次。(a) 非模态感知划分将不同模态的阶段放在同一个流水线段中, 造成段内不平衡。(b) 分离式划分为每种模态分配独立流水线段, 消除段内不平衡。(c) 模态感知数据分批进一步将微批次拆分为更小、特定于模态的子微批次, 从而平衡不同流水线段之间的工作负载。

5B 模型 (§2.2)，仅这一策略相较最佳混合划分方案就能将性能提高 13.1%。

② 通过模态感知分批消除段间不均衡。分离模态之后，还必须平衡各模态流水线段之间的工作负载。我们的第二项原则是将微批次内的数据划分为更小、特定于模态的子微批次（图 5c）。例如，如果整个视觉编码器阶段慢于某个 LM 阶段，就可以用更小的子微批次处理图像。这样一来，编码器可以执行多个更短的阶段，并可调整每个编码器阶段的延迟，使其更好地匹配单个 LM 阶段的延迟。这种模态感知分批方法可缩小整条流水线上的延迟差异，实现更为全局均衡且高效的调度。

分解式、可扩展的调度搜索。调度搜索问题可以表述为一个整体式整数线性规划 (ILP) 问题，并使用现成求解器求解 [34, 41]。这种方法精确，却难以用于实时场景，求解通常需要数分钟甚至数小时 [16]。为满足一次训练迭代的严格时间预算（通常为 10–60 秒，图 8b），我们采用分治策略，将复杂搜索问题分解为一系列更简单、更易处理的子问题。如图 7 所示，搜索器在一个三阶段循环中反复迭代，并在每一步应用专门设计的启发式方法：

① 流水线段重排 (§5.1)：首先确定流水线段的最优处理顺序。我们使用蒙特卡洛树搜索 (MCTS) 高效探索庞大的排列空间，找出有潜力的流水线段顺序。

② 流水线阶段交错 (§5.2)：在流水线段顺序固定后，再排列相应的前向和反向流水线阶段。一个快速的双队列贪心算法对这些阶段进行交错，以尽量减少流水线气泡并紧密地填充调度。

③ 逐层内存优化 (§5.3)：最后，在调度结构固定后，分别优化每个流水线段的内存使用。该阶段为所有模型层选择节省内存的策略（例如激活检查点与卸载），以尽量减小数据动态性引起的内存使用波动，并优化端到端延迟。

整个循环可以在不同 CPU 线程之间高度并行，从而确保搜索过程能随可用 CPU 资源扩展，并服务于大型分布式训练集群。

3.2 系统 workflow

上述原则由 DIP 训练规划器统一协调。该规划器由三个主要组件组成：模态感知划分器 (§4)、流水线调度搜索器 (§5) 和训练仿真器 (§6.1)。

如图 6 所示，整体 workflow 分两个时期进行。模态感知划分器包含一个离线模型块划分器和一个在线子微批次划分器。训练开始前，模型块划分器根据我们的分离式划分原则将 LMM 划分为多个模型块，并把它们分布到指定 GPU 上。训练期间，所有模型块都静态放置在各自的 GPU 工作进程上，底层网络拓扑（例如 NCCL 连接）也随之固定。每次迭代中，规划器执行以下四阶段过程：

① 元数据预取：规划器获取下一个批次的元数据（例如 token 数和图像数）。

② 自适应微批次划分：子微批次划分器根据这些元数据，把微批次拆分为更小、特定于模态的子微批次，以实现段间负载均衡。

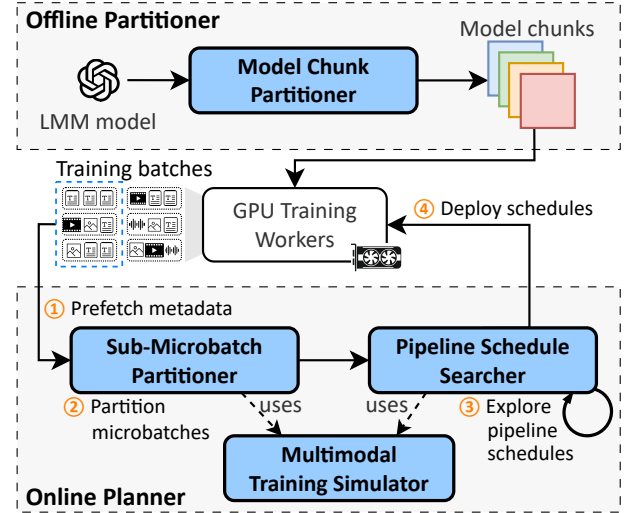


Figure 6. DIP 的整体 workflow。训练开始前，模型块划分器将 LMM 划分为模型块。每次训练迭代期间，DIP 异步执行一个四阶段在线规划过程。

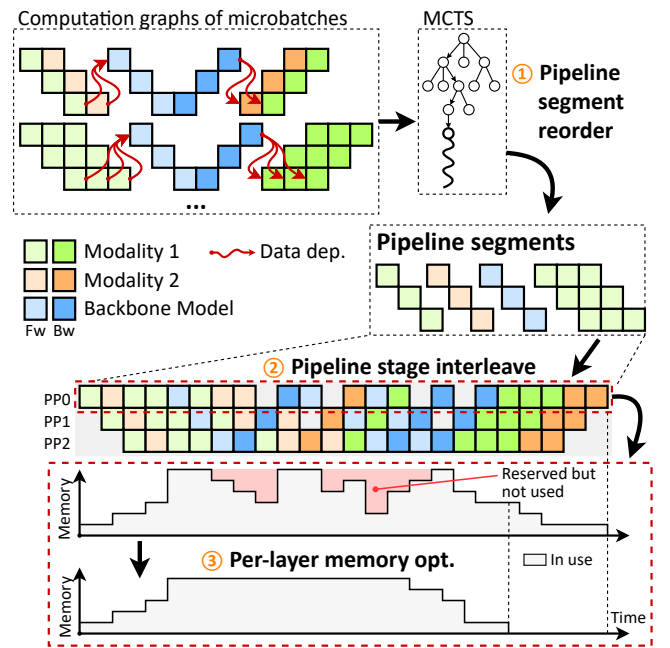


Figure 7. DIP 流水线调度搜索器的工作流。

③ 流水线调度搜索：流水线调度搜索器以仿真器给出的性能估计为指导，运行三阶段算法，为生成的子微批次寻找最优调度。

④ 运行时部署：最佳调度被编译为执行计划，并下发到分布式训练运行时执行。

4 模态感知划分器

在 §3.1 所述原则的基础上，我们提出模态感知划分。该方法在训练开始前将每个模态模块划分为多个模型块，

并在训练期间将微批次动态拆分为特定于模态的子微批次。具体而言，对于第 i 个模态，DIP 确定两个参数：子微批次大小 B_i 和流水线段数量 K_i 。因此，第 i 个模态模块被划分为 $P \cdot K_i$ 个模型块，并分布在 P 个流水线秩上。确定子微批次大小。DIP 通过系统化性能分析，为每个模态模块独立选择可行的最小子微批次大小 B_i 。较小的子微批次虽然能实现粒度更细的流水线划分和更高的调度效率，但过小的批次也可能导致 GPU 利用不足（图 9）。为权衡二者，DIP 测量每个模态模块在不同子微批次大小下的计算延迟。最优 B_i 被确定为满足以下条件的最小子微批次大小：其 GPU 计算效率至少达到全部测试配置中峰值效率的 95%。

划分模型块。确定子微批次大小后，DIP 对每个模态模块进行划分，以实现全局延迟均衡。给定 n 个模态模块，其计算延迟按 $T_1 \leq T_2 \leq \dots \leq T_n$ 排序（均在各自的 B_i 大小下测得），DIP 按这些延迟的比例分配流水线段数量。具体而言，最快的模块（ T_1 ）被配置为单个流水线段，第 i 个模块则被划分为 $K_i = \lceil T_i/T_1 \rceil$ 个流水线段。对于包含 L_i 层的模态模块，DIP 将其各层分布到 $P \cdot K_i$ 个模型块中，每个模型块包含 $L_i/(P \cdot K_i)$ 个连续层。

构造子微批次。训练执行期间，DIP 根据传入微批次的内容，为每个模态模块动态构造子微批次。对于分配给第 i 个模态模块、包含 N_i 个实例的微批次，DIP 将其均匀拆分为 $M_i = \lceil N_i/B_i \rceil$ 个子微批次。对于该模态模块，此过程总计生成 $2M_i \cdot K_i$ 个流水线段（同时计入前向和反向计算）。

5 流水线调度搜索器

5.1 基于 MCTS 的流水线段重排

DIP 通过为流水线段分配调度优先级，确定它们之间的相对顺序。这些优先级随后用于流水线阶段交错步骤 (§5.2)。为高效探索可能的优先级分配空间，DIP 采用蒙特卡洛树搜索 (MCTS) [9]。

搜索树构建。给定 n 个流水线段，MCTS 通过依次为每个位置 i （从 1 到 n ）选择流水线段，构造一个长度为 n 的序列。位于第 i 个位置的流水线段获得优先级 $n-i$ ，从而确保越早选中的段具有越高优先级。

该算法构建一棵搜索树，其中深度 d 处的每个节点对应序列中的第 d 个元素。因此，任何终止于深度 d 节点的根到节点路径，都表示前 d 个位置的一个部分序列。每个节点 v 保存其在 MCTS 树中的后代曾达到的最高性能分数 s_v 。

搜索循环。MCTS 反复执行以下四个阶段，直至收敛：

1. 节点选择：MCTS 从根节点出发，选择使置信上界 (UCB) 分数 $s_v^u + \beta \sqrt{(\log N_x)/N_v}$ 最大的子节点并向下游遍历。其中， x 表示当前节点， v 表示候选子节点， N_x 和 N_v 分别为访问次数， α 、 β 为超参数。该过程持续到到达叶节点 u 。
2. 树扩展：向搜索树加入一个表示 u 之后下一个元素的新节点，随后将 u 设为新建节点。
3. 随机回放：执行多次回放（例如 10 次试验），随机把流水线段分配到 u 之后的剩余位置，从而生成完整

序列。每个序列随后经过流水线阶段交错 (§5.2) 和逐层内存优化 (§5.3)，以计算性能分数（即端到端迭代时间）。

4. 分数反向传播：所有试验中的最佳回放分数从 u 向上传播，以更新各祖先节点的性能分数。

该循环持续执行，直到预设时间预算耗尽或整个搜索空间均被探索。

优化。我们观察到，同一微批次中处理同一种模态的流水线段具有相同的流水线结构和相近的阶段延迟。因此，它们之间的相对顺序不会影响端到端性能。利用这一洞察，DIP 可以为这类流水线段分配相同优先级并强制采用固定顺序，从而显著缩小搜索空间。

5.2 贪心流水线阶段交错

为各流水线段分配优先级后，DIP 使用双队列算法自适应地交错前向和反向流水线阶段。当可调度的前向和反向阶段同时存在时，DIP 模仿 Megatron-LM 节省内存的“一次前向、一次反向” (1F1B) 模式 [24, 39]。当任一类型不可用时，则使用贪心策略填充流水线气泡，构造紧凑调度。

初始化。DIP 首先通过训练仿真器获取所有阶段的延迟和内存消耗，并采用 §5.3 确定的最节省内存方案。这一步为随后的逐层内存优化确保足够的优化空间。

对于每个流水线秩，DIP 维护：(1) 最近已调度阶段的结束时间 (t_{last} ，初始为零)；(2) 两个按优先级降序存放前向和反向阶段的优先队列 (Q_{fw} 、 Q_{bw})；以及 (3) Q_{fw} 和 Q_{bw} 中各阶段的最小开始时间 (t_{min})。每个阶段都维护一个可调度的最小开始时间 t_{start} ：无前驱阶段时初始化为零，否则初始化为 $+\infty$ 。

迭代式调度。DIP 迭代选择一个流水线秩，并从该秩调度一个阶段：

1. 选择 t_{min} 最小的流水线秩。
2. 将 Q_{fw} 中阶段的最小开始时间 t_{fw} 和 Q_{bw} 中阶段的最小开始时间 t_{bw} 与 t_{last} 比较。
3. 若 $t_{\text{fw}} < t_{\text{last}}$ 且 $t_{\text{bw}} < t_{\text{last}}$ ，则根据最近已调度阶段的计算类型交替调度前向或反向阶段，以模拟 1F1B 模式。
4. 否则，选择 t_{start} 最小的阶段，以尽量减小 t_{min} 与该阶段开始时间之间的流水线气泡。

选中的阶段从队列中出队，并在该流水线秩上完成调度。随后更新 t_{last} 、 t_{min} ，以及所有后继阶段的 t_{start} 。

内存约束。在整个调度过程中，DIP 实时跟踪各流水线秩的内存消耗。当某个秩超过内存容量时，其前向队列会暂时停用，以防止内存溢出。

5.3 逐层内存优化

DIP 为所有模型层自适应选择适当的内存优化策略（例如激活检查点）。由于阶段交错方案已由双队列算法 (§5.2) 预先确定，该阶段可独立优化每个流水线秩的端到端延迟。

离线候选生成。训练开始前，DIP 为每个模型层枚举所有可能的优化策略，并使用训练仿真器估计其执行时间

和内存消耗。系统随后将每个前向阶段与其对应的反向阶段组合成一个阶段对。对于每个阶段对，DIP 通过以下三步，从逐层策略的组合空间中选出至多 S 个候选策略（例如 $S = 10$ ）：(1) 找出最快候选；(2) 找出最节省内存的候选；(3) 将这两个极值之间的内存范围均匀划分为 $S - 2$ 个桶，再通过多选背包算法 [1] 选出每个桶中时间效率最高的候选。

ILP 表述。对于每个流水线秩，DIP 使用 ILP 求解器选择最优候选。给定 n 个阶段对、每个阶段对有 S 个候选策略，令 s_i 、 t_i 表示第 i 个阶段对的开始和结束时间戳， $\text{lat}_{i,j}$ 、 $\text{mem}_{i,j}$ 分别表示第 i 个阶段对的第 j 个候选的阶段延迟和内存消耗。其表述使用：

- 变量： $o_{i,j} \in \{0, 1\}$ ，表示是否为第 i 个阶段对选择第 j 个候选。
- 目标：最小化总延迟 $\sum_i \sum_j o_{i,j} \cdot \text{lat}_{i,j}$ 。
- 选择约束：每个阶段对恰好选择一种策略，即 $\sum_j o_{i,j} = 1$ ($\forall 1 \leq i \leq n$)。
- 内存约束：在任意时刻 s_k ，均满足内存上限 M ，即 $\sum_i [s_i \leq s_k \leq t_i] \sum_j o_{i,j} \cdot \text{mem}_{i,j} \leq M$ ($\forall 1 \leq k \leq n$)，其中 $[\cdot]$ 为 Iverson 括号 [21]。

优化。与整体式端到端 ILP 表述相比，问题规模虽然已经显著缩小，但求解一个 ILP 实例仍可能需要数秒。我们引入两项技术实现高效求解（单个 CPU 核心上每个实例 < 10 ms）：(1) 使用贪心初始解对 ILP 求解器进行热启动；(2) 允许较小的最优性间隙（例如 $\leq 5\%$ ）以提前终止，因为弥合最后 5% 的差距收益递减，却需要高昂的计算成本。

5.4 时间复杂度分析

本节概述 DIP 训练规划器的算法结构，并分析其时间复杂度。规划器以迭代搜索循环运行，直至预设时间预算耗尽。每次迭代包含三个阶段：流水线段重排 (§5.1)、流水线阶段交错 (§5.2)，以及逐层内存优化 (§5.3)。为分析单次搜索迭代的复杂度，令 p 表示流水线秩数量， n 表示子微批次数量， S 表示候选节省内存策略的数量。此外，令 $\text{ApproxILP}(m, k)$ 表示近似 ILP 求解器在具有 $O(m)$ 个变量和 $O(k)$ 个约束的实例上的时间复杂度。

首先，流水线段重排以 $O(n)$ 时间生成所有子微批次的顺序。其次，流水线阶段交错使用的贪心调度算法，其运行时间与流水线阶段总数成正比，即 $O(p \cdot n)$ 。最后，逐层内存优化求解一组小型近似 ILP 实例，为每个流水线秩选择最优配置。每个 ILP 实例需要创建 $n \cdot S$ 个指示变量 ($o_{i,j}$)，以及 n 个选择约束和 n 个内存约束。因此，逐层内存优化的时间复杂度为 $O(p \cdot \text{ApproxILP}(n \cdot S, n))$ 。综合这些步骤，单次搜索迭代的总体时间复杂度为 $O(p \cdot (n + \text{ApproxILP}(n \cdot S, n)))$ 。

相比之下，完整 ILP 基线方法由于涉及规模巨大的变量和约束，本质上效率低下。第一，它必须全局求解整条流水线，而不是按流水线秩分解问题。第二，为防止每个流水线秩内的阶段执行重叠，它需要 $O(n^2)$ 个顺序约束；DIP 则通过高效的显式调度避免了这一开销。此外，在内存优化阶段，假设每个流水线阶段包含 L

Table 2. 评估中使用的模型规格。

名称	层数	嵌入 维度	FFN 隐藏 维度	注意力 头数	注意力 组数
ViT 5B [12]	63	1792	15360	16	16
ViT 22B [12]	48	6144	24576	48	48
Llama3 8B [17]	32	4096	14336	32	8
Qwen2 32B [51]	64	5120	27648	40	8
Qwen2 72B [51]	80	8192	29568	64	8
DiT 5B [35]	28	3584	10240	28	28
DiT 30B [35]	48	6144	24576	48	48

Table 3. 评估中使用的模型组合。

名称	模型配置	TP	PP	GPU 数
VLM-S	ViT 5B + Llama3 8B	4	4	16
VLM-M	ViT 5B + Qwen2 32B	8	4	32
VLM-L	ViT 22B + Qwen2 72B	8	8	64
T2V-S	Llama3 8B + DiT 5B	4	4	16
T2V-L	Qwen2 32B + DiT 30B	8	8	64

层、每层有 c 个候选策略，则每个阶段的搜索空间会扩张至 c^L 个候选。因此，基线 ILP 方法的复杂度上升为 $O(\text{ApproxILP}(p \cdot n \cdot c^L, p \cdot n^2))$ 。我们在 §7.4 和图 12 中对该基线与 DIP 训练规划器的性能进行了实证比较。

6 实现

DIP 以 Megatron-LM [39] 的扩展形式实现，其中包含一个与 Megatron-LM 运行时协同工作的中央规划器。规划器反复预取训练元数据、搜索流水线调度，并将优化后的调度部署到 GPU 集群。

6.1 训练仿真器

DIP 使用算子级分析建模进行性能预测。仿真器构建包含两类节点的有向无环图 (DAG)：(1) 表示底层 GPU 操作（例如矩阵乘法和集合通信）的算子节点；(2) 对应数据缓冲区（例如模型参数）的张量节点。每个节点被分配到特定设备 (GPU、CPU 或 NVLink)，并通过依赖边相连。

在延迟估计中，每个算子节点由以下量刻画：浮点运算次数 N_{fop} 、以字节计的内存访问量 N_{mem} ，以及以字节计的网络传输量 N_{net} 。给定设备能力（以 FLOPS 计的计算能力 F 、以字节/秒计的内存带宽 B_{mem} ，以及以字节/秒计的网络吞吐量 B_{net} ），延迟按下式计算： $\max\{\alpha_{\text{fop}} N_{\text{fop}} / F, \alpha_{\text{mem}} N_{\text{mem}} / B_{\text{mem}}, \alpha_{\text{net}} N_{\text{net}} / B_{\text{net}}\}$ ，其中 α_{fop} 、 α_{mem} 和 α_{net} 是效率缩放因子。用户也可以使用自定义估计规则覆盖该成本模型。

仿真器按拓扑顺序填充算子时间戳，随后通过检查相关算子节点确定张量生命周期。利用这些生命周期，可以构建内存消耗时间线，从而估计各设备的峰值内存使用量。

6.2 并行调度搜索

DIP 在 CPU 核心上执行训练仿真器和搜索算法。它首先使用预取的元数据，对各个微批次计算图进行仿真，以获取流水线阶段延迟。调度搜索期间，多个工作进程并行探索调度空间，并共享全局 MCTS 搜索统计信息。每轮搜索结束后，这些工作进程通过互斥锁保护的操作，以原子方式更新全局 MCTS 统计信息。由于工作进程每次同步之间会执行多次回放，从而摊薄同步开销，因此竞争始终很小。为避免干扰正常训练进程，DIP 将搜索工作进程数量限制为可用 CPU 核心的至多 50%（例如一台配有 8 块 H800 GPU 的机器上使用 64 个核心）。

6.3 执行计划部署

将仿真的流水线调度部署到 GPU 集群，需要把它们编译为指定计算和通信模式的物理执行计划。

调度编译。DIP 沿用 DynaPipe [24]，定义构成流水线执行计划的动作序列：

- 流水线阶段被转换为 fw_stage/bw_stage 动作，并采用 §5.3 给出的优化策略。
- 点到点 (P2P) 通信使用与计算重叠的异步内核 (isend、irecv)。
- 根据仿真时间线插入同步动作 (wait_isend、wait_irecv)。

运行时修改。我们扩展了 Megatron-LM 的 schedules 模块以支持动态计划。每个流水线工作进程通过 RPC 从中央规划器接收一个动作列表，并按顺序执行其中的动作。连续的 P2P 内核会被组合成批处理操作，以提高效率。

7 评估

7.1 方法

模型。我们在两类主要 LMM 架构上全面评估 DIP：视觉语言模型 (VLM) 和文本到视频 (T2V) 模型 (表 2)。VLM 架构将基于 ViT 的图像编码器 (5B 和 22B 参数) [12, 14] 与语言模型骨干 (Llama3 8B [17]、Qwen2 32B/72B [50]) 相结合；T2V 架构则将语言模型编码器与基于 DiT 的扩散视频解码器 (5B 和 30B 参数) [35] 相结合。如表 3 所示，我们选择了五种不同的模型组合，其参数规模从 12B 到 94B。

数据集。我们混合使用多个多样化的开源数据集 [4, 7, 25, 30, 38, 46]，其中包括纯图文对、图文交错文档和视频-字幕对。对于 VLM，我们采用 Qwen2 VL [45] 使用的 ViT 架构。我们将所有图像缩放至 728px 分辨率，每幅图像由 ViT 视觉编码器编码为 169 个 patch token (patch_size=14, spatial_merge_size=4)。多个图文数据样本被打包成 8192 token 的序列，因此每个序列最多容纳 $\lceil 8192/169 \rceil = 48$ 幅图像。对于 T2V 模型，我们采用 MovieGen [35] 的配置，将视频转码为 16 FPS，最长时长为 16 秒。处理短视频时，我们在每个微批次中最多组合 8 个视频片段，同时确保微批次总时长低于 16 秒。

基线。我们将 DIP 与四个最先进基线系统进行比较：

- 完全分片数据并行 (FSDP) [55] 是 PyTorch 中标准的内存高效训练策略。它在数据并行工作进程之间分片模型参数、梯度和优化器状态，仅在计算当前层时通过通信收集这些数据。我们采用 PyTorch 2.8.0 中的 FSDP2，并为 Transformer 块使用 ZeRO-3 配置 (reshard_after_forward=true)，以尽量降低峰值内存使用量。
- Megatron-LM [39] 是一种广泛使用的单模态 LLM 训练框架。我们采用交错流水线并行 (VPP)，并把 LMM 各层划分为参数分布大致均衡的模型块。
- nnScaler [28] 可自动完成深度神经网络训练的并行化。由于生成单个并行化计划需要数分钟，而且更新计划必须重启训练过程，我们在训练前使用有代表性的训练工作负载预先生成静态并行化计划。为公平比较不同框架，我们在 Megatron-LM 中实现了 nnScaler 的模型块划分方案和内存优化，并将性能指标标记为“nnScaler*”。
- Optimus [15] 提出粗粒度和细粒度气泡调度，用于优化含多个编码器的多模态 LLM (MLLM) 训练。粗粒度策略在流水线层面先依次执行所有模态编码器计算，再执行骨干模型；细粒度方法则用编码器计算填充 TP 通信气泡，与我们的流水线设计正交。为聚焦于流水线调度比较，我们在 DIP 中实现了 Optimus 的粗粒度气泡调度。由于 Optimus 不支持扩散解码器，我们在 T2V 模型评估中不纳入它。

评估未包含 Spindle [47]，主要因为它针对静态多任务场景设计，要求在训练前预先定义任务。这种静态设计与现代 LMM 所需的动态、灵活输入处理能力存在根本差异。

测试平台。我们一个由 8 个节点、共 64 块 NVIDIA 80GB H800 GPU 组成的集群上开展了大规模实验。每个节点配有 128 个 CPU 核心、256GB 主机内存，以及通过 200 GB/s NVLink 互连的 8 块 H800 GPU。节点间通信使用采用 rail 优化拓扑的 8×200Gbps RoCEv2 网络。此外，我们还使用了一个较小的双节点集群与 FSDP 和 Megatron-LM 基线进行比较；每个节点配有 8 块 NVIDIA 96GB H20 GPU。

指标。

我们采用训练迭代时间和模型 FLOPs 利用率 (MFU) 作为性能指标，所有报告数值均取 10 次独立运行的平均值。

7.2 端到端性能

与 LLM 训练系统比较。

我们首先将 DIP 与 FSDP 和 Megatron-LM 进行基准比较，二者都是广泛使用的单模态 LLM 训练框架。这些实验在 16-GPU H20 集群上开展，利用每块 GPU 较大的 96GB 显存，尽量减少激活重计算开销。如表 4 所示，FSDP 仅比 Megatron-LM 慢 3%，而 DIP 相比 Megatron-LM 加速 27%。鉴于 FSDP 与 Megatron-LM 的性能差距很小，我们基于 Megatron-LM 实现其余所有基线，以确保跨框架比较公平。

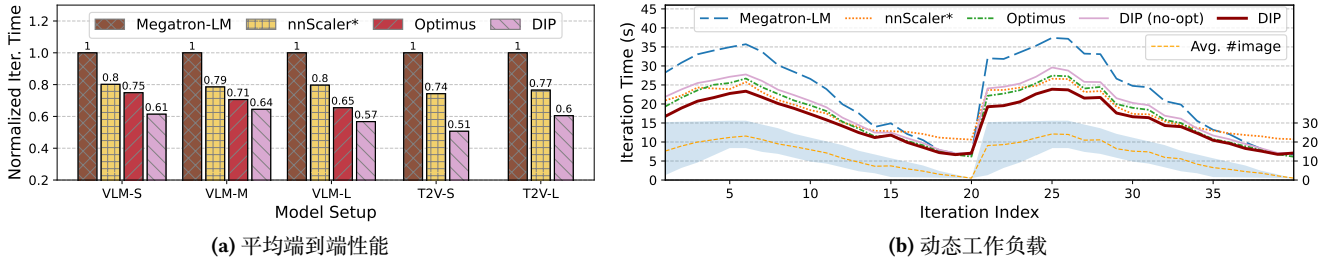


Figure 8. 端到端性能。(a) 五种模型配置在真实数据集上运行 100 次迭代的平均性能。(b) 连续 40 次迭代的端到端延迟时间线。橙色虚线表示平均图像数量。

Table 4. FSDP、Megatron-LM 和 DIP 在 16 块 H20 GPU 上的 VLM-S 端到端性能。

	FSDP [55]	Megatron-LM [39]	DIP
迭代时间 (s)	40.270	39.053	28.606
相对时间	1.03	1.00	0.73

与多模态训练系统比较。我们使用真实数据集和表 3 汇总的五种模型配置，将 DIP 的平均端到端性能与基线系统进行比较。如图 8a 所示，在 VLM 模型配置中，DIP 相比三个基线系统将训练吞吐量提高 15.6%–76.2%；在 T2V 模型配置中，相比两个基线将训练吞吐量提高 36.6%–97.3%。这说明其性能增益在不同模型架构和参数规模上保持一致。

动态工作负载。为分析动态工作负载下 DIP 相对于其他基线的性能特征，我们在 VLM-S 模型的训练迭代中手动控制图像数量上下界。我们监测了 40 次迭代，其中图像数量呈现两次“先升后降”的变化模式。每种模式包括：(1) 在上界保持为 32 的同时，将下界从 0 逐步提高到 16 (迭代 1–5)，平均图像数峰值达到 22；随后 (2) 逐步将上下界都降为零 (迭代 6–20)。

图 8b 表明，DIP 在所有系统中始终具有更优性能。在图像数量较多的阶段 (迭代 1–10)，Megatron-LM 在模态模块和数据微批次之间出现显著的计算不均衡，第 6 次迭代时相较 DIP 慢 52.9%。尽管 nnScaler 和 Optimus 都在一定程度上缓解了动态不均衡的影响，它们仍比 DIP 慢 10.4%。当图像数量和上下界之差均减小 (迭代 11–20) 时，训练工作负载逐渐接近纯语言任务，DIP 与基线系统之间的性能差距随之缩小。值得注意的是，nnScaler 仅限使用 1F1B 调度，因而必须将所有模态模块划分到同一个流水线段中；当图像编码器工作负载减小时，这会造成显著的流水线不均衡，使迭代 15–20 的性能下降 50.5%。

7.3 性能消融

性能分解。在 VLM-S 模型配置上，我们将四个关键组件 (模态感知划分器、流水线阶段交错、流水线段重排和逐层内存优化) 依次集成到 Megatron-LM 上。如表 5 所示，仅模态感知划分器就比 Megatron-LM 基线提高 17.3% 的

Table 5. DIP 各项优化的量化影响。

技术	迭代时间 (s)	$\Delta\%$
原始 Megatron-LM	26.13	0.0%
+ 模态感知划分器 (§4)	22.27	17.3%
+ 流水线阶段交错 (§5.2)	18.81	38.9%
+ 流水线段重排 (§5.1)	17.61	48.3%
+ 逐层内存优化 (§5.3)	16.05	62.8%

性能，凸显了它在 LMM 训练中的关键作用。在流水线调度搜索器的各项优化中，相较 Megatron-LM 的默认流水线机制，流水线阶段交错带来显著的 21.6% 性能提升。之后，流水线段重排和逐层内存优化通过智能重排流水线段和自适应选择内存优化策略，又将性能提高 23.9%。在图 8b 中，我们直观比较了动态工作负载下模态感知划分器和流水线调度搜索器带来的改进；其中标记为“DIP (no-opt)”的曲线排除了流水线调度搜索器的优化，用作二者分界。

子微批次大小的影响。我们使用 VLM-S 模型，测试从 4 到 32 的图像子微批次大小，并分别求得最佳和最差¹流水线调度，以研究特定于模态的子微批次大小对流水线调度和 GPU 执行效率的影响。

对图 9 的分析揭示了两个关键发现。第一，较小的子微批次大小 (4–12) 将最佳与最差调度之间的性能差距从 15.4% 显著缩小到 5.1%。方差收窄表明系统对调度配置的敏感性下降，从而降低了得到最优流水线调度的难度。第二，过小的子微批次 (<8) 由于 GPU 计算能力利用不足而出现收益递减，与中等大小的批次相比，其迭代时间增加 12.1%。通过实证验证，我们发现子微批次大小为 12 时，流水线调度灵活性与硬件利用效率达到最佳平衡。

逐层内存优化的影响。我们分析了 VLM-M 模型训练期间第一个流水线段的内存使用时间线。如图 10 所示，基线系统未能充分利用可用 GPU 内存。Megatron-LM 在稳定的 1F1B 流水线阶段表现出显著的内存波动；Optimus 则会先执行全部模态编码器计算并存储大量激活内存，因

¹我们反转流水线调度搜索器的优化目标、改为最大化迭代时间，从而得到最差流水线调度。

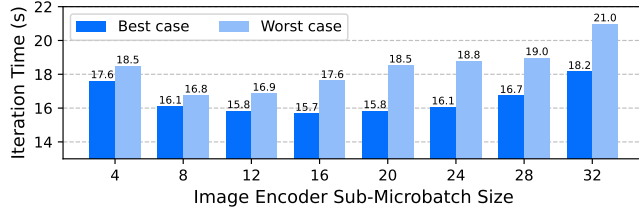


Figure 9. 子微批次大小的影响。

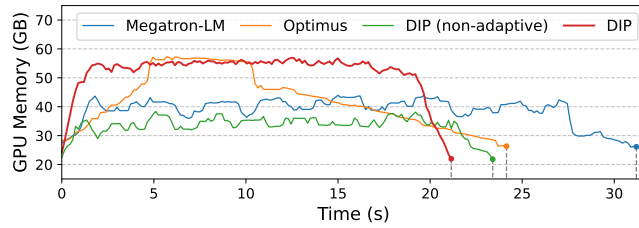


Figure 10. VLM-M 训练中第一个流水线段的内存使用时间线。“DIP (non-adaptive)”禁用了逐层内存优化，因而未使用全部可用 GPU 内存。

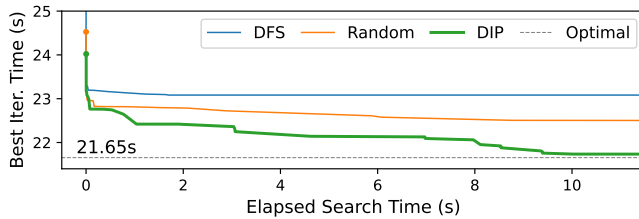


Figure 11. 不同探索策略的搜索进展。

而内存使用逐步增加。这使其峰值内存消耗比 Megatron-LM 高 25.3%。相比之下，DIP 将微批次拆分为更小的子微批次，使前向和反向流水线阶段能够以更细粒度交错，从而在整个训练迭代中始终保持较低的内存使用量。逐层内存优化进一步智能选择内存优化配置，以充分利用可用 GPU 内存；相较 Megatron-LM，其内存波动减少 52.9%，并在峰值内存相同的条件下比 Optimus 获得 12.2% 的性能增益。

7.4 规划器评估

搜索效率。为展示 DIP 流水线调度搜索的效率，我们跟踪当前最佳流水线调度性能随搜索时间的变化，并与两个分别以深度优先搜索（DFS）和随机探索替代 MCTS 算法的变体进行比较。以最大的 VLM-L 模型作为目标工作负载，我们在 64 个 CPU 核心上运行所有搜索算法变体。图 11 显示，DIP 在 10 秒内即可获得接近最优的流水线调度性能；这一搜索过程可以与 VLM-L 通常长达 20 秒的训练迭代重叠。相比之下，DFS 和随机探索策略缺少像 MCTS 那样以性能分数为引导的搜索优化，无法快速找到最优流水线调度。

搜索可扩展性。我们使用 VLM-S 和 T2V-S 两种模型配置评估搜索可扩展性。具体而言，我们测量微批次数量增

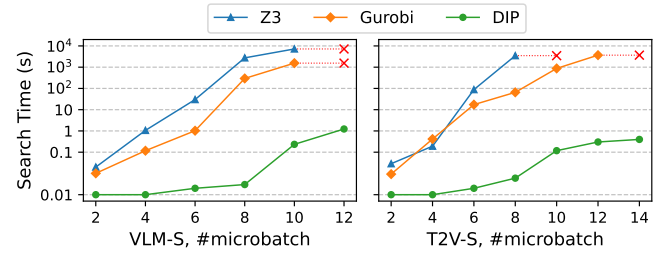


Figure 12. 在 VLM-S 和 T2V-S 上，Z3、Gurobi 与 DIP 面对不同微批次数量时的搜索时间比较。Y 轴采用对数尺度。红色叉号 (x) 表示超时 (>3 小时)。

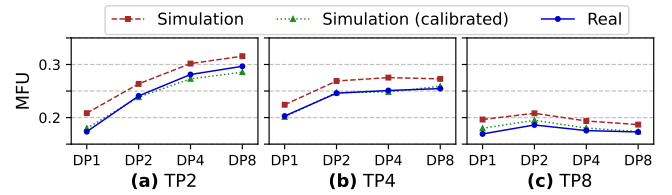


Figure 13. 校准前后的仿真结果与实际 GPU 执行结果的比较。

加时 DIP 的搜索延迟；微批次增加会相应增加流水线阶段数量和流水线调度复杂度。我们将 DIP 与两个最先进 SMT/ILP 求解器 Z3 [13] 和 Gurobi [34] (13.0 版) 进行基准比较。在实验设置中，DIP 和 Gurobi 最多使用 64 个 CPU 线程；Z3 本身不支持多线程求解，因此仅使用单线程。如图 12 所示，DIP 始终将搜索时间保持在 10 秒以内。相比之下，Z3 和 Gurobi 的搜索时间均随微批次数量呈指数增长。因此，当微批次数量超过 10 时，两个求解器都无法在 30 分钟内完成求解，不适合用于 LMM 训练中的动态流水线调度生成。

仿真准确率。我们通过 VLM-M 在 64 块 GPU 上的网格搜索实验评估 DIP 训练仿真器的准确率，并将仿真结果与实际 GPU 执行结果进行比较。我们系统评估了 DP、PP、TP 大小的所有有效组合，其中各数值均为 2 的幂，且 $TP \leq 8$ 。如图 13 所示，最优并行配置为 DP8、TP2、PP4，MFU 达到 29.7%。尽管 DIP 的默认仿真设置与真实测量值相比，初始相对误差最高可达 10%，仿真器仍成功预测出了最优并行配置。通过离线微基准校准矩阵乘法和集合通信操作的效率缩放因子，训练仿真器达到平均 97.6% 的仿真准确率。

7.5 大规模仿真

为验证 DIP 在大规模 H100 GPU 集群环境中的有效性，我们使用训练仿真器，针对两个规模庞大的大多模态模型（VLM-XL 和 T2V-XL）开展实验，评估其相对于基线方法的理论改进。两个模型和 GPU 集群的详细配置见表 6。

结果。图 14 中的实验结果表明，DIP 在 VLM-XL 和 T2V-XL 上的 MFU 分别达到 0.36 和 0.39。该系统始终优于基线方法，最高提升 82.8%；在流水线并行规模较大时优势

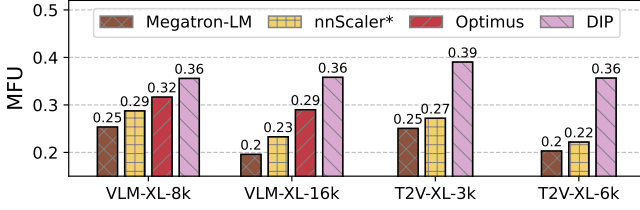


Figure 14. H100 集群上的大规模仿真。

尤为明显，因为更大的 PP 维度会引入更复杂的流水线结构，需要流水线调度搜索器对各阶段进行细致协调。

8 相关工作

自动训练并行化。已有多个系统被提出，用于自动设计深度学习模型训练的并行化策略 [28, 43, 56]。这些系统会在 DP、PP 和 TP 维度上进行全尺度模型规划。尽管它们使用穷举搜索算法生成近似最优的训练配置，规划过程往往需要大量时间，通常数分钟才能完成 [28]。LMM 训练需要动态、自适应的流水线调度，因此此类方法并不实用。

多模态模型训练系统。研究者已开发许多系统优化 [15, 19, 22, 42, 47, 53, 57]，以应对多模态模型训练中的架构异构性和训练数据异构性。例如，DistMM [19] 处理传统、基于 CLIP 的模型中的模型与数据异构性；在这类模型中，所有编码器并行运行。然而，该设计无法泛化到 LMM，因为 LMM 的编码器、骨干和解码器之间存在顺序依赖。类似地，Spindle [47] 和 Optimus [15] 都以预先定义训练任务的多模态 LLM 为目标，忽略了 LMM 训练流水线的固有动态性。具体而言，Spindle 针对静态多任务设置而设计，所有任务都必须在训练开始前预先定义。这与现代 LMM 所需的动态、灵活输入处理能力形成对比，而后者正是 DIP 关注的重点。

流水线并行调度。除 Megatron-LM 的 1F1B 调度外，研究者还提出了多种替代流水线调度。

这些方法包括 Chimera、Hanayo、zero-bubble pipeline 和 GraphPipe [23, 26, 29, 36]。Chimera [26] 引入双向流水线以减少流水线气泡；zero-bubble pipeline [36] 则将反向计算解耦为输入梯度和权重梯度两个阶段，从而进一步减少气泡。然而，这些方法假定流水线阶段延迟固定，因此仍会受到动态不平衡问题影响。GraphPipe [23] 面向包含异构模块的模型，将计算组织为有向无环图 (DAG) 而非顺序阶段，从而支持更灵活的调度。DIP 的流水线调度搜索器可以扩展，以纳入 GraphPipe 一类自定义调度。

可变序列长度流水线。若干系统处理单模态 LLM 训练中的文本序列长度变化 [5, 24, 48, 52, 54]。例如，DynaPipe [24] 使用动态规划优化多任务训练的微批次构造。WLB-LLM [48] 提出细粒度的逐文档分片策略，以均衡上下文并行组中的工作负载。FlexPipe [54] 通过在线弹性机制动态调整流水线规模，将处理短序列的 GPU 工作进程重新分配去协助处理较长序列。尽管这些方法

Table 6. 大规模仿真中使用的模型组合。

名称	模型配置	DP	TP	PP	GPU 数
VLM-XL-8k	ViT 22B + GPT 175B	128	8	8	8192
VLM-XL-16k	ViT 22B + GPT 175B	128	8	16	16384
T2V-XL-3k	Qwen2 72B + DiT 30B	96	8	4	3072
T2V-XL-6k	Qwen2 72B + DiT 30B	96	8	8	6144

对文本模型有效，它们无法处理多模态数据。相比之下，我们的方法在单条流水线内支持多种模态。

9 结论

由于流水线阶段不平衡和训练数据动态性，高效训练大多模态模型颇具挑战。本文提出DIP，通过自适应模态感知划分和高效的流水线调度搜索，解决 LMM 训练中的动态不平衡。实验结果表明，相比现有最先进训练系统，DIP 可将训练吞吐量最高提升 97.3%。

Acknowledgments

我们衷心感谢论文 shepherd Jongsoo Park 和匿名审稿人提出的深刻建议。本工作得到国家自然科学基金 (编号 62372287 和 U24A20235) 的部分资助。Mi Zeyu (yzmizeyu@sjtu.edu.cn) 为通讯作者。

References

- [1] R. D. Armstrong, D. S. Kung, P. Sinha, and A. A. Zoltners. 1983. A Computational Study of a Multiple-Choice Knapsack Algorithm. *ACM Trans. Math. Softw.* 9, 2 (June 1983), 184–198. doi:10.1145/357456.357458
- [2] Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, et al. 2025. Qwen2.5-VL Technical Report. arXiv:2502.13923 [cs.CV] <https://arxiv.org/abs/2502.13923>
- [3] Minwoo Byeon, Beomhee Park, Haecheon Kim, Sungjun Lee, Woonhyuk Baek, et al. 2022. COYO-700M: Image-Text Pair Dataset. <https://github.com/kakaobrain/coyo-dataset>.
- [4] Lin Chen, Xilin Wei, Jinsong Li, Xiaoyi Dong, Pan Zhang, et al. 2024. ShareGPT4Video: Improving Video Understanding and Generation with Better Captions. arXiv:2406.04325 [cs.CV] <https://arxiv.org/abs/2406.04325>
- [5] Qiaoling Chen, Shenggui Li, Wei Gao, Peng Sun, Yonggang Wen, et al. 2025. SPPO: Efficient Long-Sequence LLM Training via Adaptive Sequence Pipeline Parallel Offloading. arXiv:2503.10377 [cs.DC] <https://arxiv.org/abs/2503.10377>
- [6] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. 2020. A Simple Framework for Contrastive Learning of Visual Representations. In *Proceedings of the 37th International Conference on Machine Learning (ICML '20)*. JMLR.org, Virtual conference, Article 149, 11 pages.
- [7] Xiaowei Chi, Yatian Wang, Aosong Cheng, Pengjun Fang, Zeyue Tian, et al. 2024. MMTrail: A Multimodal Trailer Video Dataset with Language and Music Descriptions. arXiv:2407.20962 [cs.CV] <https://arxiv.org/abs/2407.20962>
- [8] Yew Ken Chia, Liying Cheng, Hou Pong Chan, Chaoqun Liu, Maojia Song, et al. 2024. M-Longdoc: A Benchmark For Multimodal Super-Long Document Understanding And A Retrieval-Aware Tuning Framework. arXiv:2411.06176 [cs.CL] <https://arxiv.org/abs/2411.06176>

- [9] Rémi Coulom. 2007. Efficient Selectivity and Backup Operators in Monte-Carlo Tree Search. In *Computers and Games*, H. Jaap van den Herik, Paolo Ciancarini, and H. H. L. M. (Jeroen) Donkers (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 72–83.
- [10] DeepMind. 2025. Gemma 3. <https://blog.google/technology/developers/gemma-3/>.
- [11] DeepSeek-AI, Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, et al. 2025. DeepSeek-V3 Technical Report. arXiv:2412.19437 [cs.CL] <https://arxiv.org/abs/2412.19437>
- [12] Mostafa Dehghani, Josip Djolonga, Basil Mustafa, Piotr Padlewski, Jonathan Heek, et al. 2023. Scaling Vision Transformers to 22 Billion Parameters. arXiv. arXiv:2302.05442 [cs.CV] <https://arxiv.org/abs/2302.05442>
- [13] Z3 developers. 2025. The Z3 Theorem Prover. <https://github.com/Z3Prover/z3>.
- [14] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, et al. 2021. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=YicbFdNTTy>
- [15] Weiqi Feng, Yangrui Chen, Shaoyu Wang, Yanghua Peng, Haibin Lin, et al. 2025. Optimus: Accelerating Large-Scale Multi-Modal LLM Training by Bubble Exploitation. In *2025 USENIX Annual Technical Conference (USENIX ATC 25)*. USENIX Association, Boston, MA, USA, 161–178.
- [16] Swapnil Gandhi, Mark Zhao, Athinagoras Skiadopoulos, and Christos Kozyrakis. 2024. ReCycle: Resilient Training of Large DNNs using Pipeline Adaptation. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles* (Austin, TX, USA) (SOSP '24). Association for Computing Machinery, New York, NY, USA, 211–228. doi:10.1145/3694715.3695960
- [17] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, et al. 2024. The Llama 3 Herd of Models. arXiv:2407.21783 [cs.AI] <https://arxiv.org/abs/2407.21783>
- [18] Ailin Huang, Boyong Wu, Bruce Wang, Chao Yan, Chen Hu, et al. 2025. Step-Audio: Unified Understanding and Generation in Intelligent Speech Interaction. arXiv. arXiv:2502.11946 [cs.CL] <https://arxiv.org/abs/2502.11946>
- [19] Jun Huang, Zhen Zhang, Shuai Zheng, Feng Qin, and Yida Wang. 2024. DISTMM: Accelerating Distributed Multimodal Model Training. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*. USENIX Association, Santa Clara, CA, 1157–1171. <https://www.usenix.org/conference/nsdi24/presentation/huang>
- [20] Minbin Huang, Yanxin Long, Xinchu Deng, Ruihang Chu, Jiangfeng Xiong, et al. 2024. DialogGen: Multi-modal Interactive Dialogue System for Multi-turn Text-to-Image Generation. arXiv. arXiv:2403.08857 [cs.CV] <https://arxiv.org/abs/2403.08857>
- [21] Kenneth E. Iverson. 1962. A Programming Language. In *Proceedings of the May 1-3, 1962, Spring Joint Computer Conference* (San Francisco, California) (AIEE-IRE '62 (Spring)). Association for Computing Machinery, New York, NY, USA, 345–351. doi:10.1145/1460833.1460872
- [22] Insu Jang, Runyu Lu, Nikhil Bansal, Ang Chen, and Mosharaf Chowdhury. 2025. Cornstarch: Distributed Multimodal Training Must Be Multimodality-Aware. arXiv. arXiv:2503.11367 [cs.DC] <https://arxiv.org/abs/2503.11367>
- [23] Byungsoo Jeon, Mengdi Wu, Shiyi Cao, Sunghyun Kim, Sunghyun Park, et al. 2025. GraphPipe: Improving Performance and Scalability of DNN Training with Graph Pipeline Parallelism. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1* (Rotterdam, Netherlands) (ASPLOS '25). Association for Computing Machinery, New York, NY, USA, 557–571. doi:10.1145/3669940.3707220
- [24] Chenyu Jiang, Zhen Jia, Shuai Zheng, Yida Wang, and Chuan Wu. 2024. DynaPipe: Optimizing Multi-Task Training through Dynamic Pipelines. In *Proceedings of the Nineteenth European Conference on Computer Systems* (Athens, Greece) (EuroSys '24). Association for Computing Machinery, New York, NY, USA, 542 – 559. doi:10.1145/3627703.3629585
- [25] Hugo Laurençon, Lucile Saulnier, Léo Tronchon, Stas Bekman, Amanpreet Singh, et al. 2023. OBELICS: An Open Web-Scale Filtered Dataset of Interleaved Image-Text Documents. arXiv. arXiv:2306.16527 [cs.IR] <https://arxiv.org/abs/2306.16527>
- [26] Shigang Li and Torsten Hoefler. 2021. Chimera: Efficiently Training Large-Scale Neural Networks with Bidirectional Pipelines. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis* (St. Louis, Missouri) (SC '21). Association for Computing Machinery, New York, NY, USA, Article 27, 14 pages. doi:10.1145/3458817.3476145
- [27] Zhimin Li, Jianwei Zhang, Qin Lin, Jiangfeng Xiong, Yanxin Long, et al. 2024. Hunyuan-DiT: A Powerful Multi-Resolution Diffusion Transformer with Fine-Grained Chinese Understanding. arXiv. arXiv:2405.08748 [cs.CV] <https://arxiv.org/abs/2405.08748>
- [28] Zhiqi Lin, Youshan Miao, Quanlu Zhang, Fan Yang, Yi Zhu, et al. 2024. nnScaler: Constraint-Guided Parallelization Plan Generation for Deep Learning Training. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*.
- [29] Ziming Liu, Shenggan Cheng, Haotian Zhou, and Yang You. 2023. Hanayo: Harnessing Wave-Like Pipeline Parallelism for Enhanced Large Model Training Efficiency. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis* (Denver, CO, USA) (SC '23). Association for Computing Machinery, New York, NY, USA, Article 56, 13 pages. doi:10.1145/3581784.3607073
- [30] Pan Lu, Swaroop Mishra, Tony Xia, Liang Qiu, Kai-Wei Chang, et al. 2022. Learn to Explain: Multimodal Reasoning via Thought Chains for Science Question Answering. In *The 36th Conference on Neural Information Processing Systems (NeurIPS)*.
- [31] Guoqing Ma, Haoyang Huang, Kun Yan, Liangyu Chen, Nan Duan, et al. 2025. Step-Video-T2V Technical Report: The Practice, Challenges, and Future of Video Foundation Model. arXiv. arXiv:2502.10248 [cs.CV] <https://arxiv.org/abs/2502.10248>
- [32] OpenAI. 2024. GPT-4o System Card. <https://openai.com/index/gpt-4o-system-card/>.
- [33] OpenAI. 2025. Introducing 4o Image Generation. <https://openai.com/index/introducing-4o-image-generation/>.
- [34] Gurobi Optimization. 2025. Gurobi. <https://www.gurobi.com/>.
- [35] Adam Polyak, Amit Zohar, Andrew Brown, Andros Tjandra, Animesh Sinha, et al. 2024. Movie Gen: A Cast of Media Foundation Models. arXiv. arXiv:2410.13720 [cs.CV] <https://arxiv.org/abs/2410.13720>
- [36] Penghui Qi, Xinyi Wan, Guangxing Huang, and Min Lin. 2024. Zero Bubble Pipeline Parallelism. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=tuzTNOeIO5>
- [37] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, et al. 2021. Learning Transferable Visual Models From Natural Language Supervision. arXiv. arXiv:2103.00020 [cs.CV] <https://arxiv.org/abs/2103.00020>
- [38] Christoph Schuhmann, Romain Beaumont, Richard Vencu, Cade Gordon, Ross Wightman, et al. 2022. LAION-5B: An Open Large-Scale Dataset for Training Next Generation Image-Text Models. In *Proceedings of the 36th International Conference on Neural Information Processing Systems* (New Orleans, LA, USA) (NIPS '22). Curran Associates Inc., Red Hook, NY, USA, Article 1833, 17 pages.
- [39] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, et al. 2020. Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism. arXiv. arXiv:1909.08053 [cs.CL] <https://arxiv.org/abs/1909.08053>
- [40] Shezheng Song, Xiaopeng Li, Shasha Li, Shan Zhao, Jie Yu, et al. 2025. How to Bridge the Gap between Modalities: Survey on Multimodal

- Large Language Model. arXiv. arXiv:2311.07594 [cs.CL] <https://arxiv.org/abs/2311.07594>
- [41] HiGHS Team. 2025. HiGHS: Linear Optimization Software. <https://github.com/ERGO-Code/HiGHS>.
- [42] Ye Tian, Zhen Jia, Ziyue Luo, Yida Wang, and Chuan Wu. 2024. DiffusionPipe: Training Large Diffusion Models with Efficient Pipelines. In *Proceedings of Machine Learning and Systems*.
- [43] Colin Unger, Zhihao Jia, Wei Wu, Sina Lin, Mandeep Baines, et al. 2022. Unity: Accelerating DNN Training Through Joint Optimization of Algebraic Transformations and Parallelization. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*.
- [44] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, et al. 2017. Attention is All You Need. In *Conference on Neural Information Processing Systems (Long Beach, California, USA) (NIPS'17)*. Curran Associates Inc., Red Hook, NY, USA, 6000–6010.
- [45] Peng Wang, Shuai Bai, Sinan Tan, Shijie Wang, Zhihao Fan, et al. 2024. Qwen2-VL: Enhancing Vision-Language Model's Perception of the World at Any Resolution. arXiv. arXiv:2409.12191 [cs.CV] <https://arxiv.org/abs/2409.12191>
- [46] Yi Wang, Yinan He, Yizhuo Li, Kunchang Li, Jiashuo Yu, et al. 2024. InternVid: A Large-Scale Video-Text Dataset for Multimodal Understanding and Generation. arXiv. arXiv:2307.06942 [cs.CV] <https://arxiv.org/abs/2307.06942>
- [47] Yujie Wang, Shenhan Zhu, Fangcheng Fu, Xupeng Miao, Jie Zhang, et al. 2025. Spindle: Efficient Distributed Training of Multi-Task Large Models via Wavefront Scheduling. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (Rotterdam, Netherlands) (ASPLOS '25)*. Association for Computing Machinery, New York, NY, USA, 1139–1155. doi:10.1145/3676641.3715992
- [48] Zheng Wang, Anna Cai, Xinfeng Xie, Zaifeng Pan, Yue Guan, et al. 2025. WLB-LLM: Workload-Balanced 4D Parallelism for Large Language Model Training. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*.
- [49] Jun Xu, Tao Mei, Ting Yao, and Yong Rui. 2016. MSR-VTT: A Large Video Description Dataset for Bridging Video and Language. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 5288–5296. doi:10.1109/CVPR.2016.571
- [50] An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, et al. 2024. Qwen2 Technical Report. arXiv. arXiv:2407.10671 [cs.CL] <https://arxiv.org/abs/2407.10671>
- [51] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, et al. 2025. Qwen2.5 Technical Report. arXiv. arXiv:2412.15115 [cs.CL] <https://arxiv.org/abs/2412.15115>
- [52] Yujia Zhai, Chengquan Jiang, Leyuan Wang, Xiaoying Jia, Shang Zhang, et al. 2023. ByteTransformer: A High-Performance Transformer Boosted for Variable-Length Inputs. In *2023 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE Computer Society, Los Alamitos, CA, USA, 344–355. doi:10.1109/IPDPS54959.2023.00042
- [53] Zili Zhang, Yinmin Zhong, Ranchen Ming, Hanpeng Hu, Jianjian Sun, et al. 2024. DistTrain: Addressing Model and Data Heterogeneity with Disaggregated Training for Multimodal Large Language Models. arXiv. arXiv:2408.04275 [cs.DC] <https://arxiv.org/abs/2408.04275>
- [54] Hairui Zhao, Qi Tian, Hongliang Li, and Zizhong Chen. 2025. FlexPipe: Maximizing Training Efficiency for Transformer-Based Models with Variable-Length Inputs. In *2025 USENIX Annual Technical Conference (USENIX ATC 25)*. USENIX Association, Boston, MA, USA, 143–159.
- [55] Yanli Zhao, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, et al. 2023. PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel. arXiv. arXiv:2304.11277 [cs.DC] <https://arxiv.org/abs/2304.11277>
- [56] Lianmin Zheng, Zhuohan Li, Hao Zhang, Yonghao Zhuang, Zhifeng Chen, et al. 2022. Alpa: Automating Inter- and Intra-Operator Parallelism for Distributed Deep Learning. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*.
- [57] Yijie Zheng, Bangjun Xiao, Lei Shi, Xiaoyang Li, Faming Wu, et al. 2025. Orchestrate Multimodal Data with Batch Post-Balancing to Accelerate Multimodal Large Language Model Training. arXiv. arXiv:2503.23830 [cs.DC] <https://arxiv.org/abs/2503.23830>
- [58] Wanrong Zhu, Jack Hessel, Anas Awadalla, Samir Yitzhak Gadre, Jesse Dodge, et al. 2023. Multimodal C4: An Open, Billion-Scale Corpus of Images Interleaved with Text. arXiv. arXiv:2304.06939 [cs.CV] <https://arxiv.org/abs/2304.06939>